

Taller 2: POO - Máquina expendedora de productos

Docente: Guillermo Andres De Mendoza Corrales

Fecha de entrega: Octubre 13 de 2026

Este taller está diseñado para estudiantes que están aprendiendo los fundamentos de la **Programación Orientada a Objetos (POO)** en Python. El objetivo es aplicar conceptos clave como **encapsulamiento**, **composición**, **manejo de excepciones** y la construcción de interfaces de línea de comandos (CLI).

Objetivo del Taller

Diseñar e implementar en Python un sistema orientado a objetos que simule el funcionamiento de una máquina expendedora de productos, permitiendo gestionar el inventario, ingresar dinero, procesar compras y devolver cambios mediante un menú interactivo en consola.

Arquitectura de Clases Sugerida

Los estudiantes deberán implementar al menos las siguientes tres clases aplicando principios de diseño modular:

1. Clase **Producto**

Representa un artículo disponible en la máquina expendedora.

- **Atributos mínimos (`__init__`):**
 - `nombre` (str): Nombre del producto (ej. "Soda", "Galletas").
 - `precio` (float): Costo unitario del producto.
 - `codigo` (str): Código alfanumérico para seleccionarlo (ej. "A1", "B2").
- **Métodos mínimos:**
 - `__str__(self)`: Retorna una representación en cadena legible del producto y su precio.

2. Clase **Ranura** (o **InventarioItem**)

Representa un compartimento dentro de la máquina que almacena un tipo específico de producto y su cantidad disponible. (*Demuestra el principio de composición*).

- **Atributos mínimos (`__init__`):**
 - `producto` (Producto): Instancia del objeto `Producto`.
 - `cantidad` (int): Unidades disponibles de ese producto en la ranura.
- **Métodos mínimos:**
 - `esta_disponible(self)`: Retorna `True` si la cantidad es mayor a 0.
 - `reducir_stock(self)`: Disminuye la cantidad en 1 si hay stock disponible.
 - `agregar_stock(self, cantidad)`: Incrementa el stock del producto.

3. Clase **MaquinaExpendedora**

Gestiona el estado global de la máquina: el saldo ingresado por el usuario, el diccionario de ranuras y la lógica de transacción.

- **Atributos mínimos (`__init__`):**
 - `ranuras` (dict): Un diccionario donde la clave es el código del producto y el valor es un objeto `Ranura`.
 - `dinero`(float): Bolsa de dinero recolectado.
- **Métodos mínimos:**
 - `cargar_ranura(self, codigo, producto, cantidad)`: Agrega o actualiza una ranura en la máquina.
 - `mostrar_inventario(self)`: Muestra en pantalla el código, nombre, precio y stock de todos los productos disponibles.
 - `ingresar_dinero(self, monto)`: Suma dinero al `saldo_actual` (validando que el monto sea positivo).
 - `seleccionar_producto(self, codigo)`: Valida si el código existe, si hay stock suficiente y si el saldo actual alcanza para comprarlo. Si es válido, entrega el producto y calcula el vuelto; de lo contrario, lanza una excepción o retorna un mensaje de error claro.
 - `devolver_cambio(self)`: Retorna el saldo restante y reinicia `saldo_actual` a 0.

4. Clase: **Monedero** (Bolsa de Dinero)

Representa el fondo de efectivo y el saldo temporal de la transacción. Se encarga de acumular el dinero ingresado, validar denominaciones aceptadas y calcular el cambio exacto.

- **Atributos mínimos (`__init__`):**
 - `saldo_actual` (float): Dinero ingresado por el usuario en la transacción en curso.
 - `denominaciones_aceptadas` (list): Lista de valores permitidos (ej. `[1000, 2000, 5000, 10000]`).
- **Métodos mínimos:**
 - `ingresar(self, monto)`: Valida si el monto pertenece a las denominaciones aceptadas y lo suma al saldo actual. Lanza una excepción si el billete o moneda no es válido.
 - `tiene_suficiente(self, costo)`: Retorna `True` si `saldo_actual >= costo`.
 - `descontar(self, costo)`: Resta el valor del producto comprado al saldo actual.
 - `devolver_cambio(self)`: Retorna el saldo restante y reinicia el saldo de la transacción a 0.

Requisitos de la Interfaz de Línea de Comandos (CLI)

El programa principal debe ejecutar un ciclo interactivo (`while True`) que presente un menú al usuario con las siguientes opciones operativas:

1. **Ver catálogo de productos:** Muestra la lista detallada de lo que ofrece la máquina.
2. **Ingresar dinero:** Permite al usuario simular el ingreso de monedas o billetes (ej. \$1.000, \$2.000, \$5.000, \$10.000).

3. **Comprar producto:** Solicita el código del producto deseado, procesa la transacción e informa si fue exitosa junto con su cambio.
4. **Cancelar y retirar dinero:** Devuelve el saldo acumulado si el usuario decide no comprar nada.
5. **Modo Administrador:** Permite reabastecer stock
6. **Salir:** Termina la ejecución del programa.

Criterios de Evaluación y Buenas Prácticas

- **Encapsulamiento:** Uso adecuado de atributos protegidos o privados donde sea necesario y métodos de acceso.
- **Manejo de Excepciones (`try-except`):** Capturar errores comunes como ingresar montos de dinero inválidos, seleccionar códigos inexistentes o intentar comprar sin saldo suficiente.
- **Validación de tipos y datos:** Asegurar que los precios y cantidades no sean negativos.
- **Documentación:** Cada clase y método principal debe incluir su respectivo *docstring* explicando su propósito, parámetros y valores de retorno.